# qBraid features we used

We built an eavesdropper alarm for a power-grid control link, using the BB84 quantum key protocol.

### Quantum hardware

IonQ, IQM, Rigetti, Quantinuum

IQM Garnet and IonQ Forte 1: 2 jobs each, clean link vs. eavesdropper

### Simulators

GPU state vector, tensor network

GPU state vector: tuned the alarm line on 30 qubits before touching hardware

### qBraid Lab

Notebooks, environments

One shared notebook for the team, with a Launch on qBraid button

Encouraged

### qBraid AI

AI Chat, Agent mode, MCP server

An agent drafted our batched circuit, and AI Chat compared device prices

### Transpiler

Qiskit, Cirq, Braket, OpenQASM

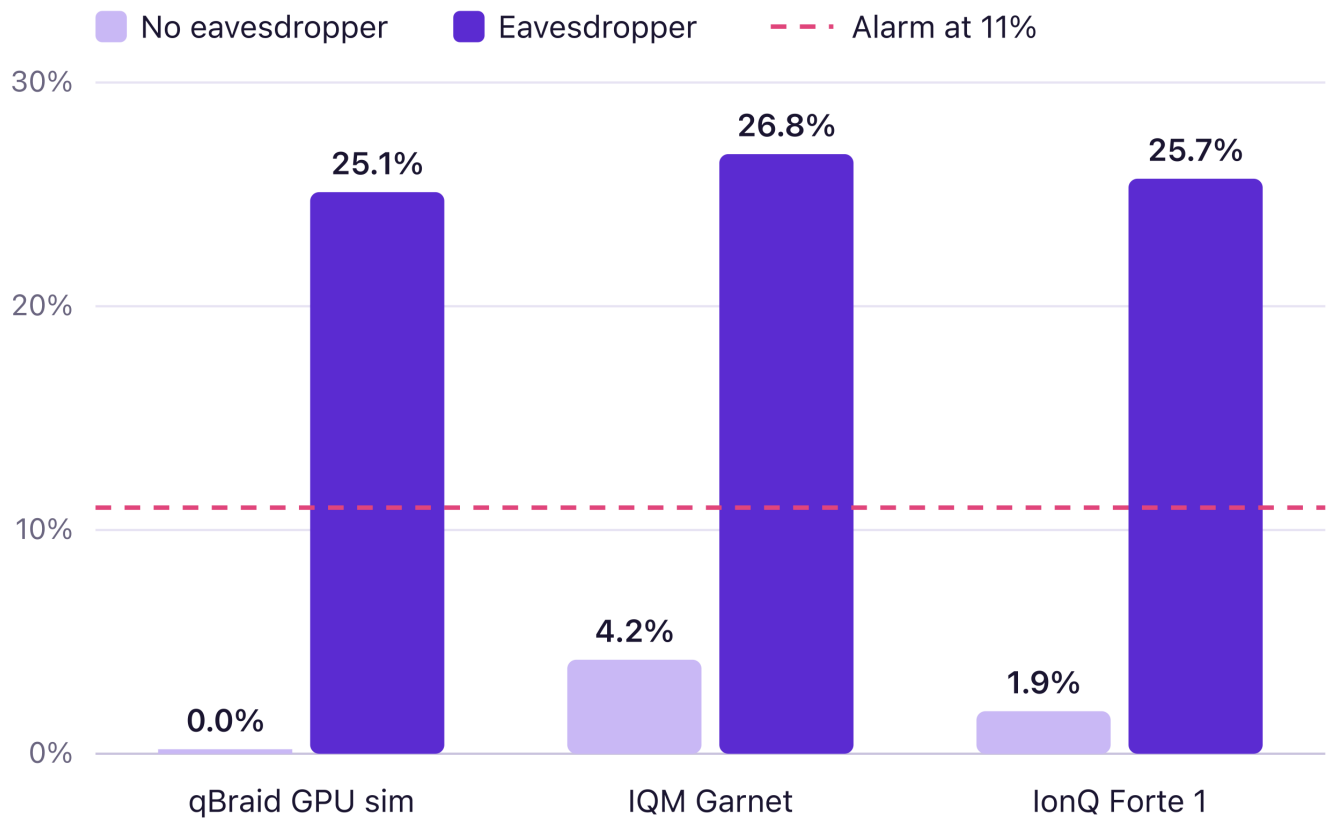
Wrote it once in Qiskit, and qBraid converted it for each device

### Jobs and results

Jobs page, job IDs, plots

Pulled job IDs from the Jobs page and plotted all three backends together

# Our alarm works on real quantum hardware




Illustrative numbers. Error rate = share of key bits that disagree.

## What it shows

Noise alone stays under our 11% alarm line. An eavesdropper pushes every backend past 25%.

## Where AI helped

An agent drafted the batched circuit in minutes. It mixed up which bits to keep, and we caught it on the simulator.

## What surprised us

IQM Garnet ran noisier than IonQ, but still left a wide margin under 11%.

# Check it

## Proof

- IQM Garnet: <ID 1>, <ID 2>
- IonQ Forte 1: <ID 3>, <ID 4>
- Job IDs copied from the Jobs page in our qBraid account

## Links

- Screen recording (1:45): <link>
- qBraid notebook: <link>
- Repo with a Launch on qBraid button: <link>

## Cost and next

- About 1,100 credits (about $11), 4% of our budget
- Batched every key bit into one circuit per run: 4 hardware jobs in total
- Next: run the alarm on Quantinuum

*Spending more credits doesn't score. A careful, cheap run beats an expensive one.*